

COGNIZANT DIGITAL NURTURE 5.0 – DEEP SKILLING PHASE

WEEK 1 : Advanced SQL Mandatory Exercises

Submitted By

Name: Dharm Bramhatimaz

Superset ID: 8000524

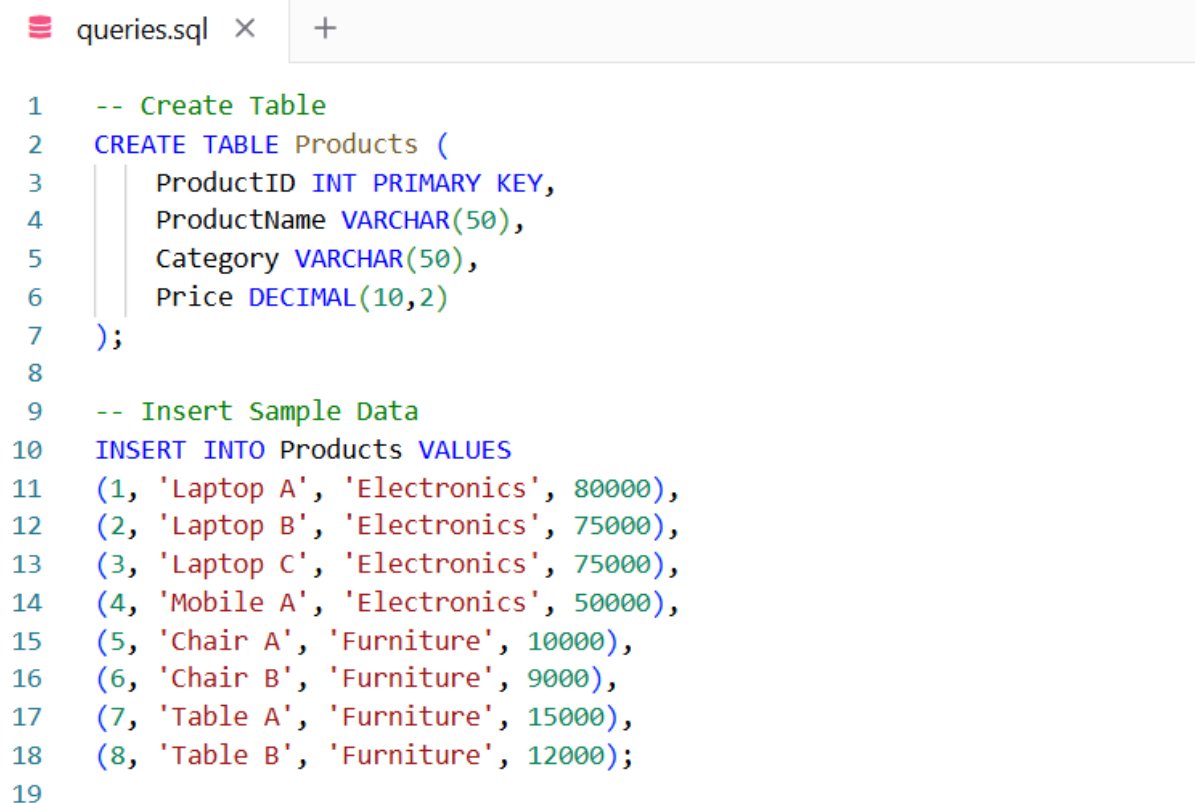
Email: dbramhatimaz20@gmail.com

University: C. V. Raman Global University, Odisha

Exercise 1: Ranking and Window Functions Goal:

Use ROW_NUMBER(), RANK(), DENSE_RANK(), OVER(), and PARTITION BY.

Scenario: Find the top 3 most expensive products in each category using different ranking functions. Steps: 1. Use ROW_NUMBER() to assign a unique rank within each category. 2. Use RANK() and DENSE_RANK() to compare how ties are handled. 3. Use PARTITION BY Category and ORDER BY Price DESC.



```
queries.sql × +
1  -- Create Table
2  CREATE TABLE Products (
3      ProductID INT PRIMARY KEY,
4      ProductName VARCHAR(50),
5      Category VARCHAR(50),
6      Price DECIMAL(10,2)
7  );
8
9  -- Insert Sample Data
10 INSERT INTO Products VALUES
11 (1, 'Laptop A', 'Electronics', 80000),
12 (2, 'Laptop B', 'Electronics', 75000),
13 (3, 'Laptop C', 'Electronics', 75000),
14 (4, 'Mobile A', 'Electronics', 50000),
15 (5, 'Chair A', 'Furniture', 10000),
16 (6, 'Chair B', 'Furniture', 9000),
17 (7, 'Table A', 'Furniture', 15000),
18 (8, 'Table B', 'Furniture', 12000);
19
```

```

20  -- Exercise 1 Solution
21  WITH RankedProducts AS
22  (
23      SELECT
24          ProductID,
25          ProductName,
26          Category,
27          Price,
28
29          ROW_NUMBER() OVER
30          (
31              PARTITION BY Category
32              ORDER BY Price DESC
33          ) AS RowNumber,
34
35          RANK() OVER
36          (
37              PARTITION BY Category
38              ORDER BY Price DESC
39          ) AS RankValue,
40
41          DENSE_RANK() OVER
42          (
43              PARTITION BY Category
44              ORDER BY Price DESC
45          ) AS DenseRankValue
46      FROM Products
47  )
48
49
50  SELECT *
51  FROM RankedProducts
52  WHERE RowNumber <= 3
53  ORDER BY Category, RowNumber;

```

Output:

ProductID	ProductName	Category	Price	RowNumber	RankValue	DenseRankValue
1	Laptop A	Electronics	80000.00	1	1	1
2	Laptop B	Electronics	75000.00	2	2	2
3	Laptop C	Electronics	75000.00	3	2	2
7	Table A	Furniture	15000.00	1	1	1
8	Table B	Furniture	12000.00	2	2	2
5	Chair A	Furniture	10000.00	3	3	3

Exercise 2: Create a Stored Procedure Goal: Create a stored procedure to retrieve employee details by department.

Steps:

1. Define the stored procedure with a parameter for DepartmentID.
2. Write the SQL query to select employee details based on the DepartmentID.
3. Create a stored procedure named `sp\_InsertEmployee` with the following code: CREATE PROCEDURE sp_InsertEmployee @FirstName VARCHAR(50), @LastName VARCHAR(50), @DepartmentID INT, @Salary DECIMAL(10,2), @JoinDate DATE

AS

```
BEGIN INSERT INTO Employees (FirstName, LastName, DepartmentID, Salary, JoinDate) VALUES (@FirstName, @LastName, @DepartmentID, @Salary, @JoinDate); END;
```



```
queries.sql x +  
  
1 CREATE TABLE Employees (  
2     EmployeeID INT AUTO_INCREMENT PRIMARY KEY,  
3     FirstName VARCHAR(50),  
4     LastName VARCHAR(50),  
5     DepartmentID INT,  
6     Salary DECIMAL(10,2),  
7     JoinDate DATE  
8 );  
9  
10 INSERT INTO Employees  
11 (FirstName, LastName, DepartmentID, Salary, JoinDate)  
12 VALUES  
13 ('Nishant', 'Kumar', 1, 50000.00, '2024-01-15'),  
14 ('Rahul', 'Sharma', 2, 60000.00, '2023-06-10'),  
15 ('Priya', 'Patel', 1, 55000.00, '2024-03-20'),  
16 ('Amit', 'Singh', 3, 70000.00, '2022-08-05');  
17  
18 DELIMITER $$  
19  
20 CREATE PROCEDURE sp_GetEmployeesByDepartment(  
21     IN p_DepartmentID INT  
22 )  
23 BEGIN  
24     SELECT *  
25     FROM Employees  
26     WHERE DepartmentID = p_DepartmentID;  
27 END $$  
28  
29 DELIMITER ;  
30  
31 CALL sp_GetEmployeesByDepartment(1);
```

Output:

EmployeeID	FirstName	LastName	DepartmentID	Salary	JoinDate
1	Nishant	Kumar	1	50000.00	2024-01-15
3	Priya	Patel	1	55000.00	2024-03-20

Exercise 5: Return Data from a Stored Procedure Goal:

Create a stored procedure that returns the total number of employees in a department.

Steps:

1. Define the stored procedure with a parameter for DepartmentID.
2. Write the SQL query to count the number of employees in the specified department.
3. Save the stored procedure by executing the Stored procedure content.

```
queries.sql × +
1  -- Create Table
2  CREATE TABLE Employees (
3      EmployeeID INT AUTO_INCREMENT PRIMARY KEY,
4      FirstName VARCHAR(50),
5      LastName VARCHAR(50),
6      DepartmentID INT,
7      Salary DECIMAL(10,2),
8      JoinDate DATE
9  );
10
11 -- Insert Sample Data
12 INSERT INTO Employees
13 (FirstName, LastName, DepartmentID, Salary, JoinDate)
14 VALUES
15 ('Nishant', 'Kumar', 1, 50000.00, '2024-01-15'),
16 ('Rahul', 'Sharma', 2, 60000.00, '2023-06-10'),
17 ('Priya', 'Patel', 1, 55000.00, '2024-03-20'),
18 ('Amit', 'Singh', 3, 70000.00, '2022-08-05'),
19 ('Rohit', 'Verma', 1, 65000.00, '2024-05-10');
20
```

```

21  -- Stored Procedure
22  DELIMITER $$
23
24  CREATE PROCEDURE sp_GetEmployeeCountByDepartment(
25  |  | IN p_DepartmentID INT
26  |  | )
27  BEGIN
28  |  | SELECT
29  |  |     DepartmentID,
30  |  |     COUNT(*) AS TotalEmployees
31  |  | FROM Employees
32  |  | WHERE DepartmentID = p_DepartmentID
33  |  | GROUP BY DepartmentID;
34  END $$
35
36  DELIMITER ;
37
38  -- Execute Procedure
39  CALL sp_GetEmployeeCountByDepartment(1);

```

Output:

```

+-----+-----+
| DepartmentID | TotalEmployees |
+-----+-----+
|           1 |           3 |
+-----+-----+

```